Matrix Arena: Adversarial Inductive Matrix Completion

Armenian LLM Summer School 2026 — Entrance Test Competition

Competition Committee

2026

Contents

1 Motivation

Matrix completion is a fundamental problem in machine learning, recommender systems, collaborative filtering, and scientific computing. The classical setting recovers a low-rank matrix from random observations. Real-world challenges, however, involve structured features, mixed generative mechanisms, and adversarial observers.

Matrix Arena combines all three:

- Structured row and column features $X \in \mathbb{R}^{n \times d}$ and $Z \in \mathbb{R}^{m \times d}$ are provided.
- The ground-truth matrix is generated from a mixture of bilinear, low-rank, and weakly nonlinear mechanisms.
- In the competitive Arena phase, participants choose which entries their opponents may observe — a strategic game-theoretic twist.

This competition tests Python fluency, numerical programming, matrix factorization, deep learning intuition, optimization, and adversarial thinking.

2 Informal Description

You receive row features X, column features Z, a partial view of a hidden interaction matrix Y^*, and a boolean mask M indicating which entries are visible. Your task is to predict the *hidden* entries of Y^*.

In the competitive phase, you also design the observation mask your opponent receives. You do so without seeing Y^* — you use only the structure of X and Z.

Both you and your opponent solve the same hidden matrix, but under different observation masks chosen by the other player.

Key rule: Participants do not create the ground-truth matrix. The grader creates one shared hidden matrix per seed. In a duel, both participants solve the same ground-truth matrix under different opponent-generated observation masks.

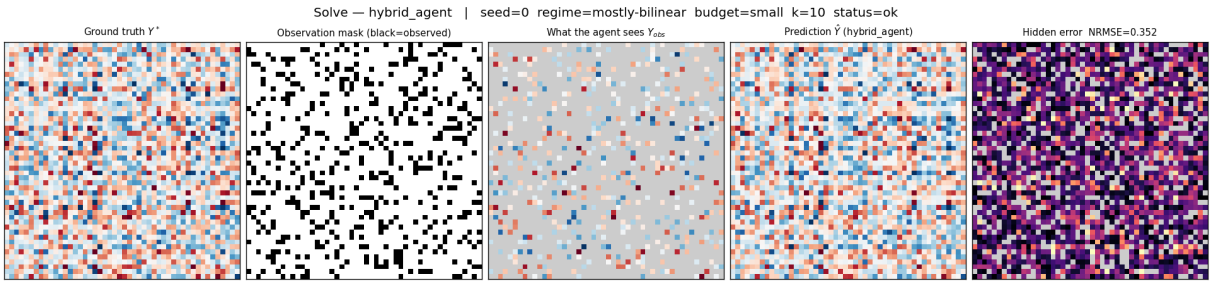

Figure 1: Anatomy of a single reconstruction (the hybrid baseline on a bilinear-regime instance). From left to right: the hidden ground truth Y^*; the observation mask M (observed cells in black); the masked input Y^{obs} the agent actually receives (visible cells colored, hidden cells gray); the agent's prediction $\hat{Y}$; and the absolute error on the *hidden* cells only — the entries that count toward the score. Signed values use a diverging red/blue colormap centered at zero. These figures are produced by `scripts/visualize_match.py` and are the standard way to read every example in this document.

3 Formal Problem Definition

3.1 Shared instance

For each seed s and budget configuration B, the grader creates:

$$(X, Z, Y^*) = \texttt{generate\_instance}(s, B) \tag{1}$$

where $X \in \mathbb{R}^{n \times d}$, $Z \in \mathbb{R}^{m \times d}$, $Y^* \in \mathbb{R}^{n \times m}$.

3.2 Observation mask

A valid mask $M \in \{0,1\}^{n \times m}$ satisfies:

$$\sum_{j=1}^{m} M_{ij} = k \quad \forall i \in [n] \tag{2}$$

$$\sum_{i=1}^{n} M_{ij} = k \quad \forall j \in [m] \tag{3}$$

and the bipartite graph G_M with row nodes $R_0, \ldots, R_{n-1}$, column nodes $C_0, \ldots, C_{m-1}$, and edge R_i–C_j iff $M_{ij} = 1$ must be *connected*.

3.3 Observed matrix

$$Y_{ij}^{\text{obs}} = \begin{cases} Y_{ij}^* & \text{if } M_{ij} = 1 \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

3.4 Participant input/output

Input to `solve`: X, Z, Y^{obs}, M, B, s
Output of `solve`: $\hat{Y} \in \mathbb{R}^{n \times m}$
Input to `attack`: X, Z, k, B, s (no access to Y^*)
Output of `attack`: a valid mask $M \in \{0,1\}^{n \times n}$

4 Matrix Generator

The ground-truth matrix is:

$$Y_{ij}^* = w_{\text{bil}}\, \mathbf{x}_i^\top W \mathbf{z}_j + w_{\text{lat}}\, \mathbf{u}_i^\top \mathbf{v}_j + w_{\text{nl}} \sum_{\ell=1}^{r_{\text{nl}}} a_\ell \tanh\Big((\mathbf{x}_i^\top \mathbf{p}_\ell)(\mathbf{z}_j^\top \mathbf{q}_\ell)\Big) + \varepsilon_{ij} \tag{5}$$

where:

- $W \in \mathbb{R}^{d \times d}$ is a random bilinear interaction matrix.
- $U \in \mathbb{R}^{n \times r}$, $V \in \mathbb{R}^{m \times r}$ give the latent low-rank term.
- $P \in \mathbb{R}^{d \times r_{\text{nl}}}$, $Q \in \mathbb{R}^{d \times r_{\text{nl}}}$, $a \in \mathbb{R}^{r_{\text{nl}}}$ define the nonlinear component.
- $\varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$ is entry-level noise.
- $w_{\text{bil}}, w_{\text{lat}}, w_{\text{nl}}$ are regime-dependent mixture weights.

Y^* is normalized to have zero mean and unit standard deviation after generation.

4.1 Regimes

The generator selects a *regime* deterministically from the seed:

Regime	w_{bil}	w_{lat}	w_{nl}	σ
Mostly bilinear	0.85	0.10	0.05	0.05
Mostly low-rank	0.10	0.85	0.05	0.05
Mixed	0.45	0.45	0.10	0.05
Nonlinear dominant	0.25	0.25	0.50	0.10
Noisy	0.60	0.30	0.10	0.20
Spiky	0.40	0.40	0.20	0.05

In the spiky regime, a random subset of rows and columns receive feature vectors with 3× larger magnitude, creating high-leverage entries.

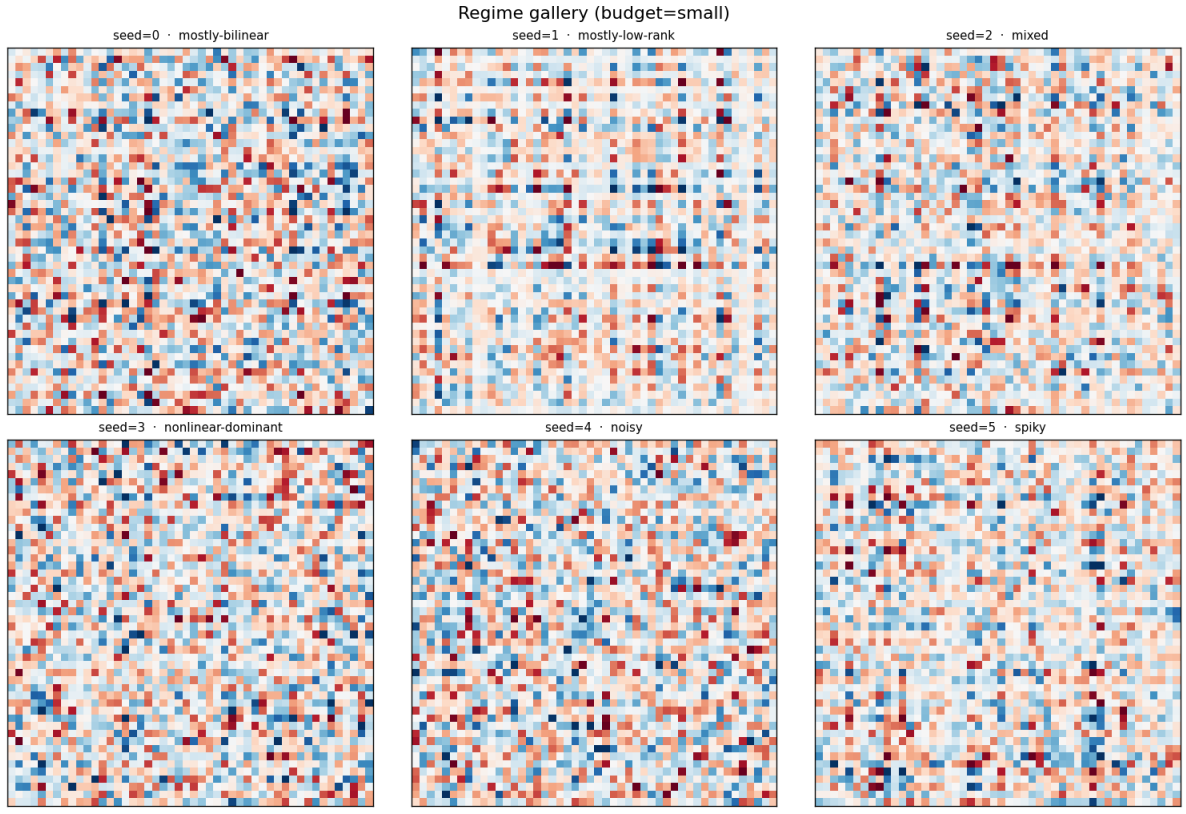

Figure 2: One normalized ground-truth matrix Y^* per generative regime (small budget, 48×48). The regimes differ in structure: the low-rank-dominant instance shows smoother row/column banding, while the bilinear, noisy, and nonlinear instances carry more high-frequency detail. A robust agent must perform well across all of them, since the regime is chosen deterministically from the hidden seed and is not revealed.

5 Solve-Only Phase

Each agent is evaluated on a set of seeds using the *official mask* (a deterministic random k-regular connected mask generated by the grader).

The reconstruction loss on hidden entries is:

$$\mathrm{NRMSE}(\hat{Y}, Y^*, M) = \frac{\sqrt{\frac{1}{|\bar{M}|}\sum_{(i,j):\, M_{ij}=0}(\hat{Y}_{ij} - Y^*_{ij})^2}}{\mathrm{std}(Y^*[\bar{M}]) + 10^{-8}} \tag{6}$$

where $\bar{M}$ denotes the complement of M (hidden entries).

The instance-level solve score is:

$$\mathrm{SolveScore} = \log\left(\frac{\mathrm{NRMSE}_{\mathrm{mean}}}{\max(\mathrm{NRMSE}_{\mathrm{agent}},\, 10^{-12})}\right) \tag{7}$$

where $\mathrm{NRMSE}_{\mathrm{mean}}$ is the loss of the mean-fill baseline. Scores are positive when the agent beats the baseline, zero if equal, negative if worse.

6 Arena Phase (Duels)

6.1 Duel protocol

A duel between agents A and B on seed s:

1. Grader creates (X, Z, Y^*) from seed s.
2. A generates mask M_B for B: $M_B \leftarrow A.\texttt{attack}(X, Z, k, B, s_A)$.
3. B generates mask M_A for A: $M_A \leftarrow B.\texttt{attack}(X, Z, k, B, s_B)$.
4. Invalid masks are replaced by the official mask; the attacking agent receives penalty $\delta = 0.5$.
5. A solves under M_A: $\hat{Y}_A \leftarrow A.\texttt{solve}(X, Z, Y_A^{\mathrm{obs}}, M_A, B, s)$.
6. B solves under M_B: $\hat{Y}_B \leftarrow B.\texttt{solve}(X, Z, Y_B^{\mathrm{obs}}, M_B, B, s)$.
7. Compute $L_A = \mathrm{NRMSE}(\hat{Y}_A, Y^*, M_A) + \delta_A$ and L_B similarly.

6.2 Outcome

$$\mathrm{winner} = \begin{cases} A & \text{if } L_A < L_B\,(1-\varepsilon) \\ B & \text{if } L_B < L_A\,(1-\varepsilon) \\ \mathrm{draw} & \text{otherwise} \end{cases} \qquad \varepsilon = 10^{-4} \tag{8}$$

7 Ranking

7.1 Arena points

$$\mathrm{ArenaPoints}_i = \frac{W_i + \frac{1}{2}D_i}{W_i + D_i + L_i} \tag{9}$$

where W_i, D_i, L_i are total wins, draws, and losses for agent i.

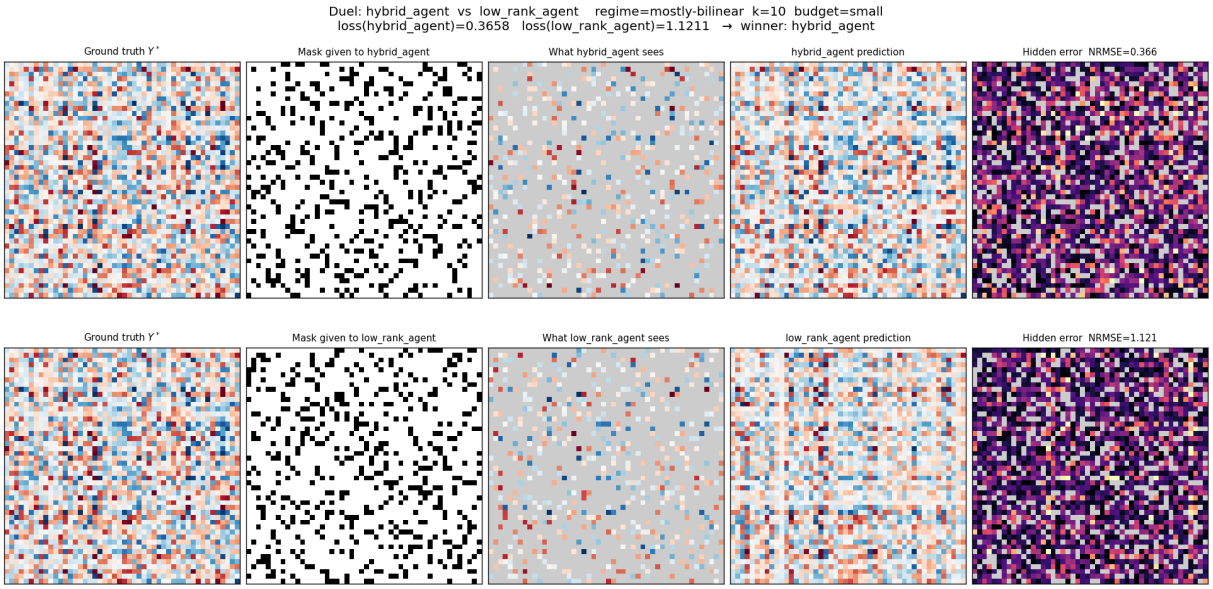

Figure 3: A duel (top row: hybrid baseline; bottom row: low-rank baseline). Both agents reconstruct the *same* hidden matrix Y^* (identical leftmost panels), but each receives a *different* observation mask chosen by its opponent (second column) — so each sees a different subset of the matrix (third column). Comparing the rightmost hidden-error panels settles the duel: the hybrid agent's error is markedly lower (NRMSE ≈ 0.37 vs. ≈ 1.12), so it wins per Eq. (8). Read each row using the panel layout of Figure 1.

7.2 Bradley-Terry rating

The Bradley-Terry model assigns skill parameters r_i such that:

$$P(i \text{ beats } j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} \tag{10}$$

Parameters are fit by the MM algorithm on observed win/draw counts. Draws count as $\frac{1}{2}$ win for each side. Ratings are normalized to zero mean.

Elo-like conversion:

$$\text{Elo}_i = 1500 + \frac{400}{\ln 10} \cdot r_i \tag{11}$$

7.3 Final score

$$\text{FinalScore}_i = 0.60 \cdot \widetilde{S}_i + 0.35 \cdot \widetilde{E}_i + 0.05 \cdot R_i \tag{12}$$

where $\widetilde{S}_i$ is min-max normalized solve score, $\widetilde{E}_i$ is min-max normalized Elo, and $R_i \in [0, 1]$ is a robustness score that penalizes crashes, invalid masks, timeouts, and NaN/Inf outputs.

If max = min during normalization, all agents receive 0.5.

8 Budget Levels

Level	n	m	d	r	r_{nl}	k	Solve timeout
small	48	48	12	4	2	10	1.0 s
medium	96	96	24	8	4	12	5.0 s
large	160	160	32	12	6	16	15.0 s

Default weighting across budget levels: small 30%, medium 45%, large 25%.
Public grader uses **small** and **medium** only. Hidden grader includes **large**.

9 Participant API

Participants submit a single Python file defining:

```
class Agent:
    def solve(self, X, Z, Y_obs, mask, budget, seed):
        """
␣␣␣␣␣␣␣␣X␣␣␣␣␣␣␣:␣np.ndarray␣(n,␣d)␣␣␣--␣row␣features
␣␣␣␣␣␣␣␣Z␣␣␣␣␣␣␣:␣np.ndarray␣(m,␣d)␣␣␣--␣column␣features
␣␣␣␣␣␣␣␣Y_obs␣␣␣:␣np.ndarray␣(n,␣m)␣␣␣--␣observed␣entries␣(0␣where␣hidden)
␣␣␣␣␣␣␣␣mask␣␣␣␣:␣np.ndarray␣(n,␣m),␣bool␣␣␣--␣True␣where␣observed
␣␣␣␣␣␣␣␣budget␣␣:␣dict␣␣␣--␣config␣info
␣␣␣␣␣␣␣␣seed␣␣␣␣:␣int

␣␣␣␣␣␣␣␣Returns
␣␣␣␣␣␣␣␣-------
␣␣␣␣␣␣␣␣Y_hat␣␣␣:␣np.ndarray␣(n,␣m)
␣␣␣␣␣␣␣␣"""

    def attack(self, X, Z, k, budget, seed):
        """
␣␣␣␣␣␣␣␣Returns
␣␣␣␣␣␣␣␣-------
␣␣␣␣␣␣␣␣mask␣␣:␣np.ndarray␣(n,␣n),␣bool
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣k-regular,␣connected␣bipartite␣adjacency␣matrix
␣␣␣␣␣␣␣␣"""
```

The `seed` argument is decoupled from the instance. The `seed` passed to `solve` and `attack` exists only so that an agent's own randomness can be made deterministic. It is *not* the seed used to generate the instance: the grader derives it from the (private) generation seed through a one-way function, and the graded evaluation draws secret, high-entropy generation seeds that are never exposed. Consequently an agent cannot rebuild Y^* by calling the public generator on `seed`, nor by brute-forcing which generation seed reproduces the fully-visible X. Such attempts fail on the real grader — they produce an unrelated matrix — and merely waste the agent's time budget.

10 Validity and Robustness Constraints

10.1 Valid predictions

A prediction $\hat{Y}$ is valid if:

- Shape equals (n, m).
- All entries are finite ($|\hat{Y}_{ij}| \leq 10^5$).
- No NaN or Inf values.

Invalid predictions receive NRMSE $= 10^6$.

10.2 Valid masks

A mask M is valid if it satisfies equations (2)–(3) and the induced bipartite graph is connected. Invalid masks receive a penalty of $\delta = 0.5$ added to the attacking agent's final loss for that duel.

10.3 Timeouts

Agents exceeding `solve_timeout_s` or `attack_timeout_s` are penalized as if they returned an invalid result.

10.4 Fair play: no instance reconstruction

The task is to *predict* the hidden entries, not to recover the instance. The following are forbidden and do not work against the graded evaluation:

- Rebuilding Y^* by calling the public generator on the provided `seed` (the seed is decoupled; see the Participant API section).
- Recovering the generation seed by brute-forcing which seed reproduces the visible X (the graded evaluation uses secret, high-entropy generation seeds, so the search space is intractable).
- Reading Y^* out of the grader's process memory. The hidden grader runs each agent in an isolated subprocess.

Agents must use only their declared inputs (X, Z, Y^{obs}, M, `budget`, `seed`) to form a prediction.

11 Suggested Tactics

11.1 Reconstruction (solve)

1. **Mean baseline**: fill all hidden entries with the observed mean. Serves as the scoring reference.
2. **Row/column means**: use per-row and per-column averages.
3. **Ridge regression**: construct features $\phi_{ij} = [\mathbf{x}_i, \mathbf{z}_j, \mathbf{x}_i \odot \mathbf{z}_j]$ and fit $Y_{ij} \approx \phi_{ij}^\top w$ via ridge regression.
4. **Bilinear regression**: directly fit $\hat{Y} = XWZ^\top$ by minimizing the observed MSE over W.
5. **Low-rank factorization**: fit $\hat{Y} \approx PQ^\top$ via alternating least squares (ALS) or gradient descent.
6. **Hybrid**: combine bilinear and low-rank components.
7. **PyTorch optimization**: use Adam or L-BFGS on a differentiable objective combining all terms (bilinear + low-rank + optional nonlinear).
8. **Ensembling**: average predictions across multiple ranks and regularization strengths.
9. **Early stopping**: hold out a subset of observed entries for validation.
10. **Robust normalization**: center and scale predictions to match observed statistics.

Note: the nonlinear component (tanh) is *weak and restricted* with at most $r_{\text{nl}} \leq 6$ terms. The task is robust prediction, not exact nonlinear recovery.

11.2 Adversarial mask design (attack)

1. **Random regular**: simplest valid attack. Provides no advantage but satisfies all constraints.
2. **Low-leverage observation**: compute row leverage scores from X and column leverage scores from Z. Force the opponent to observe entries in low-leverage feature regions, where extrapolation is harder.
3. **Minimum spanning tree mask**: design a mask whose bipartite observation graph is a spanning tree (minimal connectivity). This minimizes information flow while remaining valid.
4. **Feature-antipodal mask**: place observations where $\mathbf{x}_i \approx -\mathbf{x}_{i'}$ and $\mathbf{z}_j \approx -\mathbf{z}_{j'}$, forcing sign symmetry ambiguities.
5. **Diagonal avoidance**: avoid placing observations along the bilinear diagonal $XWZ^\top$ to prevent easy rank-1 recovery.

12 Reproducibility and Deterministic Grading

All randomness is controlled by explicit seeds via `numpy.random.default_rng`. There is no global random state. Running the grader twice on the same instances produces identical results.

The grader separates two seeds. The *generation seed* builds (X, Z, Y^*) and is private to the grader; in the graded evaluation it is drawn with high entropy and never exposed. The *agent seed* passed to `solve`/`attack` is a one-way-derived value, decoupled from the generation seed, provided solely so an agent's own randomness can be deterministic. Participants must ensure their agents are deterministic given the same agent seed; non-deterministic outputs are detected and penalized.

13 Baselines

The following reference baselines are provided in the `baselines/` directory:

- **mean_agent**: fills all hidden entries with the observed mean. Used as the scoring reference in equation (7).
- **ridge_agent**: ridge regression on $[\mathbf{x}_i, \mathbf{z}_j, \mathbf{x}_i \odot \mathbf{z}_j]$.
- **low_rank_agent**: ALS matrix factorization $\hat{Y} \approx PQ^\top$.
- **hybrid_agent**: bilinear term $XWZ^\top$ plus low-rank ALS residual.
- **random_attack_agent**: mean-fill solve with a random-regular attack mask.

A positive solve score requires beating the mean baseline. The competition leaderboard normalizes scores relative to all submitted agents and baselines.

14 Submission Instructions

1. Copy `examples/agent_template.py` to a new file, e.g. `my_agent.py`.
2. Implement `solve()` and `attack()`.
3. Test locally:

```
python scripts/run_public_grader.py \
    --agents my_agent.py --seeds 20
```

4. Verify your agent is valid:

```
pytest tests/ -v
```

5. Submit `my_agent.py` to the competition portal.

Your submission must:

- Define exactly one class named `Agent` with both `solve` and `attack` methods.
- Not read from or write to the filesystem during evaluation.
- Not rely on shared mutable state between calls.
- Return valid `numpy` arrays of the correct shape and dtype.